Capstone Project:

Agentic AI System for Automated Discharge Summaries for Hospitals

1. Project Overview

Project Description:

This project introduces an end-to-end Agentic AI-powered solution designed to optimize patient discharge management in hospitals. The system automatically analyzes discharge reports and related clinical documents, validates them against hospital Electronic Health Report (EHR) systems and clinical guidelines, and identifies potential care gaps or inconsistencies.

It leverages an Agentic Retrieval-Augmented Generation (RAG) framework with Human-in-the-Loop (HITL) feedback to enable accurate, context-aware question answering for hospital administrators and care coordinators.

Finally, the solution generates a comprehensive and standardized patient discharge summary, improving care continuity, reducing errors, and supporting informed clinical and operational decision-making.

The system orchestrates multiple specialized AI agents to extract, normalize, validate, audit, and reason over multilingual, multi-format clinical data.

Business Context:

A global multi-specialty hospital network manages a high volume of patient discharges across regions, with discharge documents received in diverse formats and multiple languages. Patient discharge documents like doctor's discharge reports, lab reports and bills, arrive from different sources and vendors in multiple formats (PDFs, scanned handwritten discharge report by doctor, Word documents, structured EHR exports) and multiple languages (English, Hindi, Spanish, German, etc.).

Manual review of discharge summaries for care continuity, medication reconciliation, follow-up instructions, and compliance with clinical pathways is time consuming, error-prone, dependent on senior clinical staff. Failures in accurate discharge auditing led to re-admissions, medication errors, missed follow-ups, regulatory non-compliance.

Proposed Solution:

Implement an end-to-end Agentic AI–powered system capable of monitoring and processing incoming clinical data across multiple formats and languages, including doctor discharge reports, laboratory results, and billing records.

The system will automatically validate the data for completeness, consistency, and adherence to defined business and clinical rules, identifying any missing or incorrect information in real time.

Upon successful validation, the platform will generate a standardized and unified electronic discharge summary report, ensuring accuracy, improving operational efficiency, and supporting safe and seamless patient care transitions.

Project Objective:

Design and implement an end-to-end Agentic AI system using AWS Bedrock LLMs (Nova Lite, Cohere Command R+) that:

- Ingest Patient discharge documents, lab reports and bills arrive from different sources and vendors in multiple formats and multiple languages
- Extracts and translates discharge documents data to English.
- Validates discharge documents line-items against a mock EHR system.
- Indexes discharge documents for RAG-based question answering.
- Generates validation reports with audit trails.
- Supports human-in-the-loop feedback and corrections.
- Generates standardized patient discharge summary report which includes Patient details, admission details, doctors' details, discharge diagnosis and Discharge Instructions, Follow-up Appointment, Discharge Prescriptions in table format with replacing medical terminology by common English like (instead of BID use twice daily) and in remarks timing also , lab reports along with vendor details, include full bill details and payment status and discharge release status.

2. Problem Statement

Design and implement an end-to-end Agentic AI system called Agentic AI System for Automated Discharge Summaries for Hospitals that automates the end-to-end lifecycle of patient discharge procedure in hospitals using open-source frameworks such as LangGraph, LangChain, LiteLLM, FastAPI, and Pydantic.

The system must:

1. Ingest discharge report from doctor, lab reports and hospital bills (PDF/DOCX/PNG) from a simulated mailbox folder.
2. Extract data includes header information, tables, and line items.

3. Translate all non-English text into English.
4. Validate the extracted data:
    - Against a mock EHR system (FASTAPI-based backend returning JSON data).
    - Against configurable validation rules for missing or inconsistent fields.
5. Generate structured validation reports (JSON + HTML/PDF) in which medical terms can be converted to common man understandable format.
6. Provide a RAG-based question-answering interface that enables a human support agent to query patient information and if out of context questions, response should be "I don't know".
7. Integrate a human-in-the-loop feedback system through a Streamlit dashboard.

All LLM-based interactions are mediated by a unified LLM Gateway (LiteLLM) connected to AWS Bedrock APIs.

3. Functional requirements

Design and implement an end-to-end Agentic AI powered solution that includes:

3.1. Clinical Document Monitoring

Design (architecture/approach only) an agent that monitors:

- A hospital inbox, EHR export folder, or secure file system
- Detects new discharge report from doctor and associated clinical documents like lab reports, hospital bills. (No live EHR integration is required—file system simulation is sufficient.)

3.2 Clinical Data Extraction (Format & Language Agnostic)

- Extract structured and unstructured clinical data from:
    - Discharge report from doctor
    - Medication lists
    - Lab reports
    - Clinical notes (typed or scanned)
    - Hospital bills
    - Support multi modal content (text, tables, images, handwriting)
    - Handling multiple languages

3.3. Clinical Language Normalization

- Translate extracted clinical content and medical terms into standard English (eg: BID means twice daily)
- Include translation confidence in output.

- Normalize:
    - Medical abbreviations
    - Drug names
    - Diagnoses into standardized clinical terminology

3.4. Clinical & Business Validation

3.4.1. Clinical Completeness Validation

Validate discharge report by doctor, lab report by vendor and bill generated by hospital against configurable rules (YAML) and check for the completeness for the following fields. If any of the following fields are missing, then auto generation of the discharge summary will be blocked and HITL will take necessary action.

- Basic details like patient id, patient name, age, gender, address
- Admission date, Discharge date, admitted ward/Bed No. details
- Consulted doctors' information
- Mandatory discharge diagnosis present
- Medication reconciliation done
- Adverse Drug Reaction/Allergy info present
- Follow-up appointment documented
- Discharge instructions present
- Follow up appointment details
- Discharge instructions
- Abnormal labs have stated actions/follow-up (simulated rule)
- High-risk meds have counseling notes (optional rule)
- Mandatory Prescription fields:
    - Sl.no
    - Medicine name
    - strength
    - dosage
    - frequency
    - Route (e.g.: ORAL)
    - period
    - Remarks
    - Total quantity

3.4.2 Cross Validation vs Mock EHR / Care Plan / Labs

Validate extracted discharge report data against:

- mock EHR patient chart
- mock treatment plan

- mock lab values
- mock medication list

Some of the Mock API calls are:

- GET /ehr/patient/{patient_id}
- GET /ehr/med_orders/{patient_id}
    - GET /ehr/labs/{patient_id}
    - GET /ehr/careplan/{patient_id}
    - GET /ehr/allergies/{patient_id}
    - GET /guidelines/pathway/{diagnosis_code} (simulated)

Validate:

- Medication omissions/conflicts (check patient's medication given in discharge report by the doctor differs from Mock EHR history for each patient)
- Allergy contradictions in prescription
- Discharge diagnosis mismatch
- Follow-up missing despite care plan requiring it
- Lab follow-up mismatch (abnormal lab unresolved)
- Discharge approval from doctor's side should be true
- Bill needs settlement before release

Each validation agent should load the configs / rules.yaml configuration to check and compliance before comparing with the EHR data, the rules.yaml file is a crucial configuration file.

3.5. Audit & Risk Report Generation

- Generate a clinician/admin friendly discharge summary highlighting:
    - Missing clinical information
    - Discrepancies between discharge summary and EHR
    - Medication conflicts or omissions
    - Translation confidence scores

Generate:

- JSON audit report (system consumption)
- HTML/PDF report (clinician/patient-friendly)

Include:

- Missing clinical information list
- Discrepancy list with evidence references
- Medication conflict/omission summary
- Translation/normalization confidence
- Patient Discharge summary auto approval risk level: Low / Medium / High
- Recommendations: Approve / Needs Review / Urgent Attention
- Audit trail: timestamps, agent decisions, tool calls, trace IDs
- Total Bill amount and payment status

3.6 Agentic RAG for Clinical Question Answering

- Provide a context-aware QA assistant for hospital admins for retrieving information.
- Example queries:
  - "Was the patient discharged with appropriate follow up for diabetes?"
  - "List medication discrepancies for Patient ID X."

3.7 RAG Must Be Implemented Using Agentic AI

The required RAG Agent Roles are:

- Indexing Agent – Parses and indexes discharge documents into a Vector DB
- Retrieval Agent – Converts clinician questions into embeddings and retrieves relevant chunks
- Augmentation Agent – Re-ranks retrieved clinical context
- Generation Agent – Generates grounded, clinically safe responses
- Reflection Agent – Evaluates response quality using RAG Triad metrics

Healthcare Agent Roles (Non RAG)

The overall solution must use Agentic AI and define the following agents:

Core Agents

- Discharge Monitor Agent
- Clinical Extractor Agent
- Clinical Translation & Normalization Agent

- Clinical Data Completeness Agent
- EHR Validation Agent
- Discharge Summary Generation Agent

Agent Tools

Each agent must use well-defined tools:

- Clinical Watcher Tool – Detects new discharge files
- Clinical Data Harvester Tool – Extracts text, tables, images
- Medical Lang Bridge Tool - Translates and normalizes clinical language
- Clinical Rules Engine Tool – Validates clinical completeness rules
- EHR Validation Tool – Cross checks with simulated EHR data
- Clinical Insight Reporter Tool – Generates audit and risk reports

3.8 **Human-in-the-Loop Dashboard:**

Create the Streamlit UI for the following functionalities:

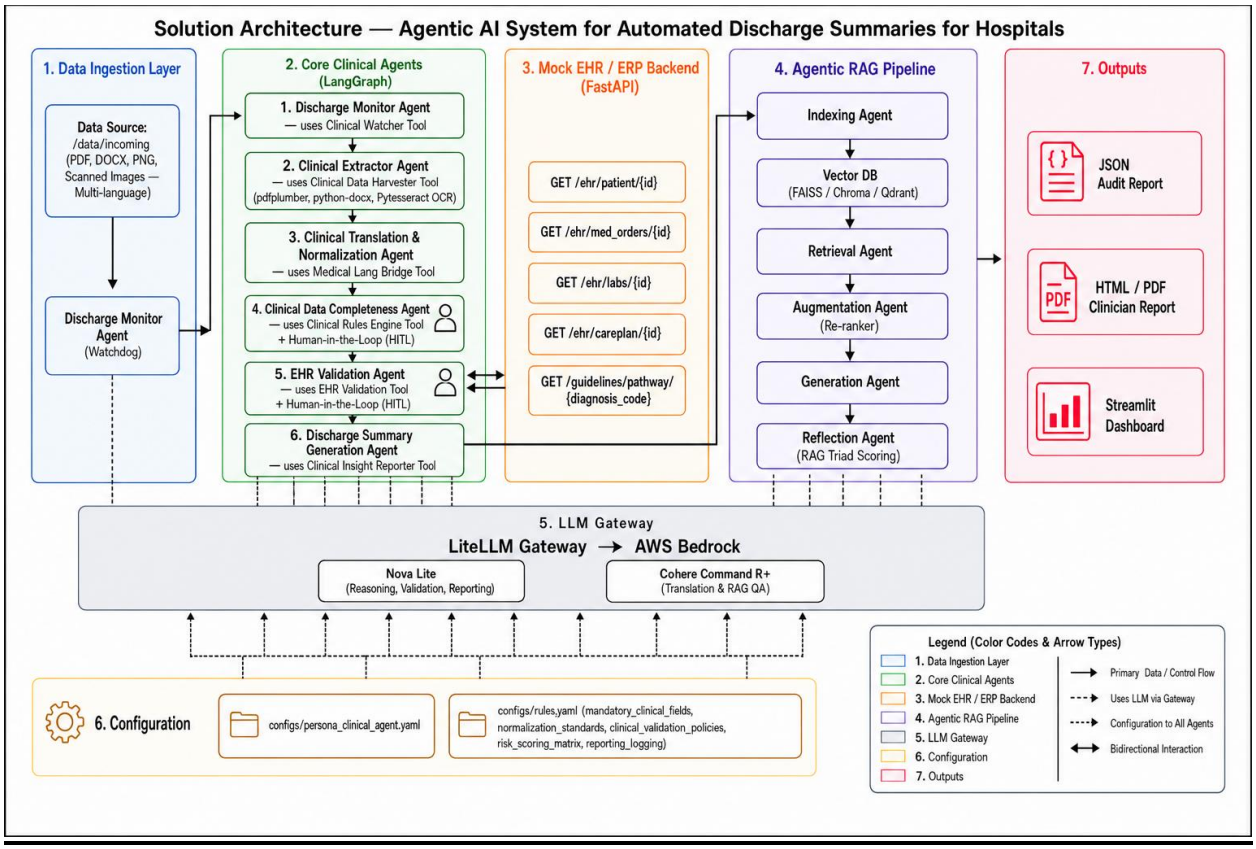
- View discharge documents + extracted structure
- View report and risk score for automatic approval
- Accept clinician corrections (med list, follow-up, diagnosis)
- Trigger "re-run validation"
- Store feedback as training data:
  - corrected fields
  - approved risk label
  - notes
- Track "resolved vs unresolved gaps" per patient
- Q&A
- Summary document generation in patient friendly language replacing medical terms in PDF/HTML/json format which includes patient's basic details, admission details, discharge report details from doctor, lab reports, prescription details and bill information for auto approved cases.

4 Non-Functional Requirements

- Performance: Efficient processing of large invoice batches
- Scalability: Modular agent design allows horizontal scaling
- Security: Secure email access and document handling

- Reliability: Robust error handling and fallback mechanisms
- Usability: Intuitive UI for support agents
- Maintainability: Modular codebase with documentation

5 Solution Architecture



6 Implementation Details

Technology Stack

Layer	Tools / Libraries
Language	Python 3.11+
Framework	LangGraph, LangChain
Backend (EHR)	FastAPI
Frontend (UI)	Streamlit
LLM Gateway	LiteLLM
OCR /image	Pytesseract
Document Parsing	pdfplumber, python-docx

Vector DB	FAISS / Qdrant/ Chroma
Schema Validation	Pydantic
Translation	AWS Bedrock Models
Prompt Templates	YAML-based Persona
Data Source	/data/incoming
RAG Evaluation	RAG Triad Scoring

AWS Bedrock Model Selection

Task	Recommended AWS	Reason
Reasoning /	Nova Lite (by AWS)	Excellent for structured reasoning,
RAG-based	Cohere Command R+	Optimized for retrieval-augmented
Language	Cohere Command R+	Natively supports multi-language

Role of LLM Gateway (LiteLLM) : Why We Need It

Since multiple agents use different models (Nova Lite, Cohere Command R+), LiteLLM acts as the centralized gateway between your Agentic System and AWS Bedrock.

Agentic System Specification

Each Agent in LangGraph performs a well-defined role, interacting through message passing. Below are updated details including which agents will use LLMs (via LiteLLM → Bedrock).

Agentic System Specification (LangGraph + Google ADK Split)

A) Core Agents (Non-RAG)

Agent	Primary Function	Framework	Uses LLM?
Discharge Monitor Agent	Detect new files + kickoff	Google ADK	No
Clinical Extractor Agent	Parse PDF/DOCX/OCR	Google ADK	Optional (for structuring only)
Translation & Normalization Agent	Translate + normalize terms	LangGraph	Yes
Clinical Data Completeness Agent	Validate mandatory rules	LangGraph	Yes (guarded)
EHR Validation Agent	Cross-check with mock HER	LangGraph	Yes (reasoning + alignment)

Discharge summary generation Agent	risk scoring + report generation	LangGraph	Yes

B) RAG Agents (Must be Agentic)

RAG Agent	Function	Framework	Uses LLM?
Indexing Agent	chunk + embed + store	LangGraph	Optional
Retrieval Agent	retrieve relevant chunks	LangGraph	No
Augmentation Agent	rerank/compress	LangGraph	Yes/optional
Generation Agent	grounded QA	LangGraph	Yes
Reflection Agent	RAG triad scoring + safety label	LangGraph	Yes

Agent & Persona Mapping

Create a .yaml file to define the personas for the clinical agents.

Agent	**Persona Role**	**Core Responsibility**
Discharge Monitor Agent	System Watchdog	Detects new discharge documents in inbox/folder, triggers workflow.
Clinical Extractor Agent	Medical Document Parser	Extracts text, tables, and images from clinical docs using OCR.
Clinical Translation & Normalization Agent	Clinical Linguist	Translates multilingual content to English and normalizes medical terms (ICD/SNOMED).

Clinical Data Completeness Agent	Clinical Auditor	Validates presence of mandatory fields (diagnosis, meds, follow-ups).
EHR Validation Agent	Cross-Reference Auditor	Compares discharge data against Mock EHR (meds, labs, treatment plans).
Discharge summary generation Agent	Compliance & Risk Reporter	Generates risk scores, summarizes discrepancies, and builds final clinical discharge summary.

configs/rules.yaml Mapping

Section	**Purpose**
mandatory_clinical_fields	Specifies required fields (patient id, patient name, discharge diagnosis, medications, follow-up instructions). Used by Data Completeness Agent.
normalization_standards	Defines expected clinical terminologies and maps abbreviations (e.g., mapping to ICD-10 or SNOMED) and translate medical terms to common English words.
clinical_validation_policies	Rules defining clinical completeness (e.g., abnormal labs must have follow-up actions documented).
risk_scoring_matrix	• Defines how to score Patient Discharge summary auto approval risk level (Low / Medium / High) • (Low/Medium/High) based on medication conflicts or missing data. • Recommendations: ○ low: "Approve — Auto-release"

	○ medium: "Needs Review for any data entry mistakes — Standard HITL" ○ high: "Urgent Attention — Block release" • Weights: ○ missing_mandatory_field: 3 ○ Address:1,Gender: 1 ○ incomplete_prescription_fields: 4 ○ medication_omission: 3 ○ medication_added: 4 ○ high_risk_med_missing_in_ehr: 9 # always escalates to High tier (pages pharmacist) ○ allergy_contradiction: 8 # always escalates ○ diagnosis_mismatch: 4 ○ followup_missing: 2 ○ abnormal_lab_unresolved: 3 ○ low_translation_confidence: 3 ○ high_risk_med_no_counseling: 4 ○ bill_unpaid_with_discharge_ok: 5 # business: bill needs settlement before release • Thresholds: ○ low_max: 2 # auto-approve if <= 2 ○ medium_max: 8 hitl ○ high: if > 8 hitl escalation to higher level
Reporting	Controls the report formats (JSON, HTML/PDF) and the output directory for clinician review.
Logging	Enables audit trails, tool calls tracking, and error tracking for debugging.

Agent vs Rules Mapping

Agent	Uses Which Rules	Purpose
Clinical Data Completeness Agent	mandatory_clinical_fields, clinical_validation_policies	To ensure clinical documents are complete and meet baseline hospital standards.
Clinical Translation & Normalization Agent	normalization_standards	To ensure standardized clinical terminology across different doctors and languages.
EHR Validation Agent	clinical_validation_policies, risk_scoring_matrix	To cross-check against EHR data and identify severe medication or care plan discrepancies.
Discharge summary generation Agent	reporting, risk_scoring_matrix	To categorize overall patient risk and generate consistently formatted clinician-friendly reports.
Discharge Monitor Agent	Logging	To record events, ingestion timestamps, and track workflow triggers.

7 Folder Structure

You can follow the below file structure or can have your own structure

```
clinical-discharge-auditor/
├── data/
│   ├── incoming/               # Discharge dataset (PDFs, DOCX, scans)
│   ├── ehr_mock_data/          # patients.json, meds.json, labs.json, 
├── mock_ehr/
│   ├── app.py                  # FASTAPI mock EHR API
│   └── data/                   # (same as ehr_mock_data/)
├── agents/
│   ├── monitor_agent.py
│   ├── extractor_agent.py
│   ├── translation_normalization_agent.py
│   ├── completeness_agent.py
│   ├── ehr_validation_agent.py
│   ├── reporting_agent.py
│   └── rag_agents/
│       ├── indexing_agent.py
│       ├── retrieval_agent.py
│       ├── augmentation_agent.py
│       ├── generation_agent.py
│       └── reflection_agent.py
├── configs/
│   ├── persona_clinical_agent.yaml
│   ├── rules.yaml
├── ui/
│   └── streamlit_app.py
├── langgraph_workflows/
│   └── clinical_auditor_workflow.yaml
└── README.md
```

Caller	Calls / Includes	Purpose
monitor_agent.py	→ extractor_agent.py	Triggers document extraction when a new clinical file is detected.
extractor_agent.py	→ translation_normalization_agent.py	Passes extracted text, tables, and OCR data.
translation_normalization_agent.py	→ completeness_agent.py	Passes translated English text and normalized medical terminology.
completeness_agent.py	→ ehr_validation_agent.py	Validates completeness and passes structured data for cross-referencing.
ehr_validation_agent.py	→ mock_ehr/app.py	Calls mock APIs to validate discharge data vs current EHR records.

ehr_validation_agent.py	→ reporting_agent.py	Sends list of discrepancies, omissions, and conflicts.
reporting_agent.py	→ rag_agents/	Passes final validation summaries to be indexed for the QA Assistant.
streamlit_app.py	→ LangGraph Workflow API	Orchestrates human-in-the-loop review, corrections, and manual re-validation.
All LLM Agents	→ configs/persona_clinical_agent.yaml	Loads specific agent persona definitions and system prompts.
Validation & Reporting Agents	→ configs/rules.yaml	Loads medical validation, completeness rules, and risk scoring criteria.

8 Deliverables

- Source code (GitHub repo -Optional if Trainees can get access to repo)
- Working demo (video or live)
- Presentation slides – not exceeding 5 slides
- Prompt templates and persona files (.yaml / json)